What is OOP?

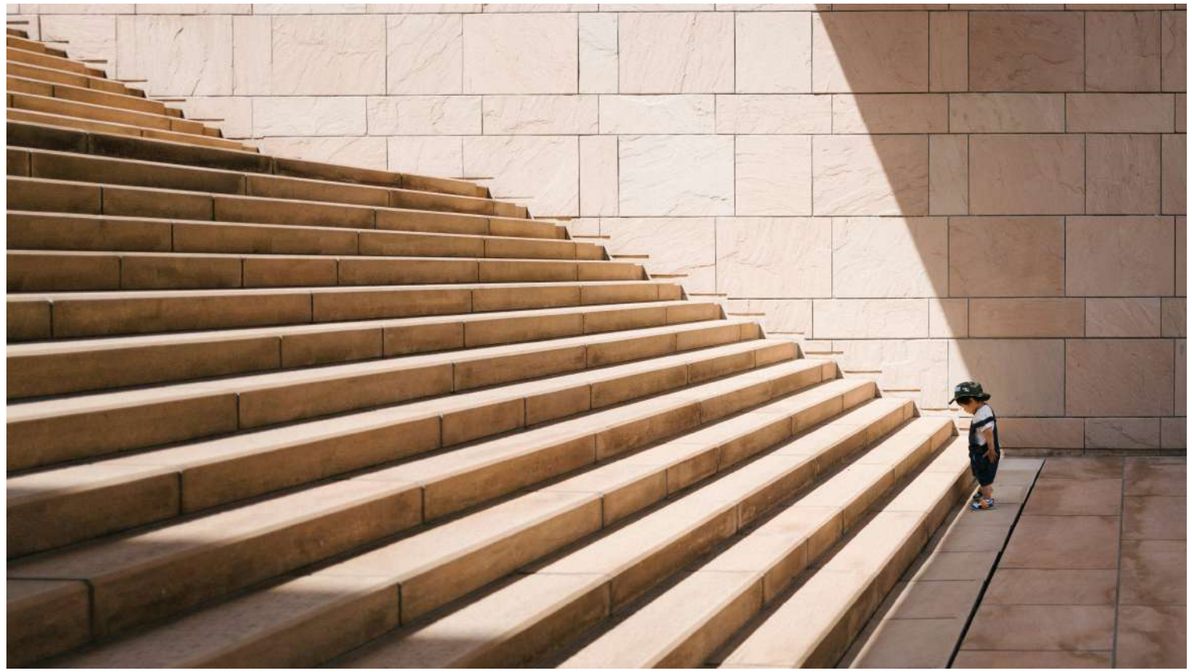
INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

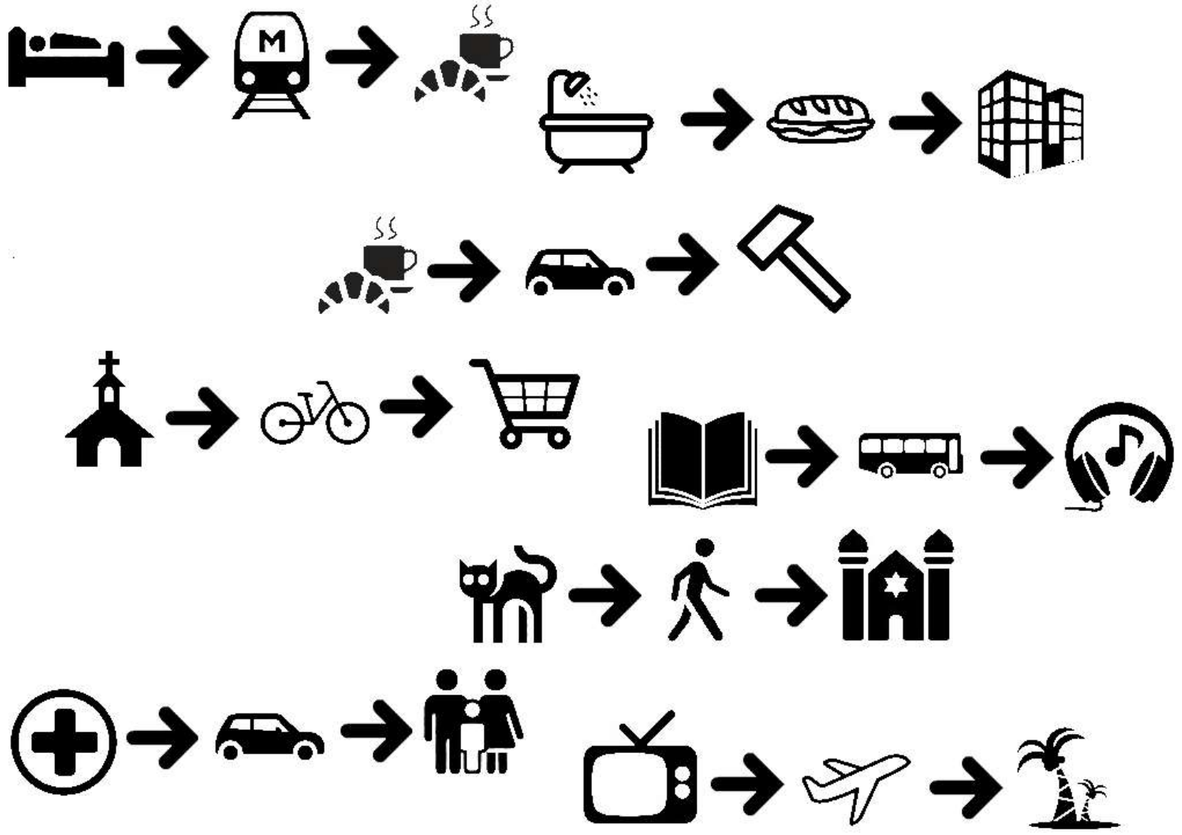
Procedural programming

- Code as a sequence of steps
- Great for data analysis



[1] Image source: https://unsplash.com/@tateisimikito

Thinking in sequences



Procedural programming

- Code as a sequence of steps
- Great for data analysis

Object-oriented programming

- ***Code as interactions of objects***
- Great for building software
- *Maintainable and reusable code!*

Objects

$$\text{Object} = \text{data} + \text{functionality}$$

email = lara@company.com
phone = 614-555-0177

place order
cancel order

State - an object's *data*

Behavior - an object's *functionality*

Objects in Python

- *Everything in Python is an object*

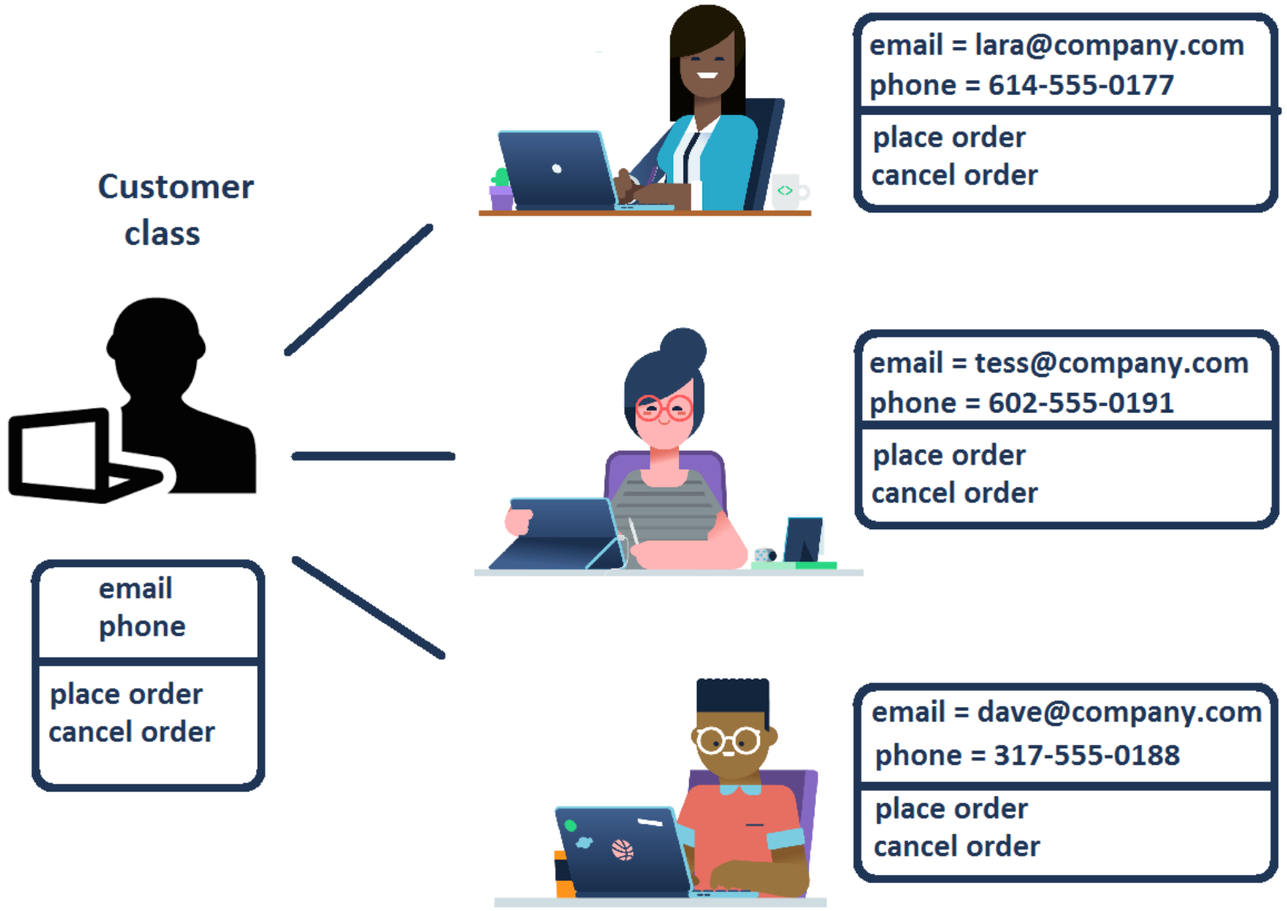
Object	Type
`5`	`int`
`"Hello"`	`str`
`pd.DataFrame()`	`DataFrame`
`sum()`	`function`
…	…

Classes as blueprints

- **Class:** a blueprint for objects outlining possible states and behaviors

Classes as blueprints

- **Class** : a blueprint for objects outlining possible states and behaviors



Classes in Python

- Python objects of the same type behave in the same way
- `lists` are a class
  - Created with comma-separated values `[1, 2, 3, 4, 5]`
  - Share the same methods, e.g., `.append()`
- Use `type()` to find the class

```
type([1, 2, 3, 4, 5])
```

```
<class 'list'>
```

Attributes and methods

State ↔ attributes

```
import pandas as pd
df = pd.DataFrame({"a": [1,2,3],
                   "b": [4,5,6]})
# shape attribute
df.shape
```

```
(3, 2)
```

- Use `obj.` to access attributes and methods

Behavior ↔ methods

```
import pandas as pd
df = pd.DataFrame({"a": [1,2,3],
                   "b": [4,5,6]})
# head method
df.head()
```

```
   a  b
0  1  4
1  2  5
2  3  6
```

Displaying attributes and methods

```
# Display attributes and methods
dir([1, 2, 3, 4])
```

```
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 ...
 'pop',
 'remove',
 'reverse',
 'sort']
```

```
# Display attributes and methods
dir(list)
```

```
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 ...
 'pop',
 'remove',
 'reverse',
 'sort']
```

Cheat sheet

Term	Definition
Class	A **blueprint**/template used to build objects
Object	A combination of *data* and *functionality*; An **instance** of a class
State	*Data* associated with an object, assigned through **attributes**
Behavior	An object's *functionality*, defined through **methods**

Let's review!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Class anatomy: attributes and methods

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

A Customer class

```
class Customer:
    # Code for class goes here
    pass
```

- `class <name>:` starts a class definition
- Code inside `class` is indented
- Use `pass` to create an "empty" class

```
c_one = Customer()
c_two = Customer()
```

- Use `ClassName()` to create an object of class `ClassName`

Add methods to a class

```
class Customer:
    def identify(self, name):
        print("I am Customer " + name)
```

- Method definition = function definition within class
- **Use `self` as the first argument in method definition**

```
cust = Customer()
cust.identify("Laura")
```

```
I am Customer Laura
```

- Ignore `self` when calling a method on an object

```
class Customer:
    def identify(self, name):
        print("I am Customer " + name)

cust = Customer()
cust.identify("Laura")
```

What is self?

- Classes are templates
- `self` should be the first argument of any method
- `self` is a stand-in for a (not yet created) object
- `cust.identify("Laura")` *will be interpreted as* `Customer.identify(cust, "Laura")`

We need attributes

- OOP bundles data with methods that operate on data
  - `Customer` 's' name should be an attribute

$$\text{Attributes are created by assignment } (=) \text{ in methods}$$

Add an attribute to class

```
class Customer:
    # Set the name attribute of an object to new_name
    def set_name(self, new_name):
        # Create an attribute by assigning a value
        # Will create .name when set_name is called
        self.name = new_name
# Create an object
# .name doesn't exist here yet
cust = Customer()
# .name is created and set to "Lara de Silva"
cust.set_name("Lara de Silva")
print(cust.name)
```

```
Lara de Silva
```

Old version

```
class Customer:



    # Using a parameter
    def identify(self, name):
        print("I am Customer" + name)
```

```
cust = Customer()

cust.identify("Eris Odoro")
```

```
I am Customer Eris Odoro
```

New version

```
class Customer:
    def set_name(self, new_name):
        self.name = new_name

    # Using .name from the object it*self*
    def identify(self):
        print("I am Customer" + self.name)
```

```
cust = Customer()
cust.set_name("Rashid Volkov")
cust.identify()
```

```
I am Customer Rashid Volkov
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Class anatomy: the __init__ constructor

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

Methods and attributes

- Methods are function definitions within a class
- `self` as the first argument
- Define attributes by assignment
- Refer to attributes in class via `self.___`
- Calling lots of methods could become unsustainable!

```
class MyClass:
    # function definition in class
    # first argument is self
    def my_method1(self, other_args...):
        # do things here
    def my_method2(self, my_attr):
        # attribute created by assignment
        self.my_attr = my_attr
        ...
```

Constructor

- Add data to object when creating it
- **Constructor** `__init__()` method is called every time an object is created
  - Called automatically because of `__methodname__` syntax

```
class Customer:
    def __init__(self, name):
        # Create the .name attribute and set it to name parameter
        self.name = name
        print("The __init__ method was called")
```

Constructor

```
# __init__ is implicitly called
cust = Customer("Lara de Silva")
print(cust.name)
```

```
The __init__ method was called
Lara de Silva
```

Attributes in methods

```
class MyClass:
    def my_method1(self, attr1):
        self.attr1 = attr1
        ...

    def my_method2(self, attr2):
        self.attr2 = attr2
        ...

obj = MyClass()
# attr1 created
obj.my_method1(val1)
# attr2 created
obj.my_method2(val2)
```

Attributes in the constructor

```
class MyClass:
    def __init__(self, attr1, attr2):
        self.attr1 = attr1
        self.attr2 = attr2
        ...
# All attributes are created
obj = MyClass(val1, val2)
```

- Generally we should use the constructor
- Attributes are created when the object is created
- *More usable and maintainable code*

Add arguments

```
class Customer:
    # Add balance argument
    def __init__(self, name, balance):
        self.name = name

        # Add the balance attribute
        self.balance = balance
        print("The __init__ method was called")
```

Add parameters

```
# __init__ is called
cust = Customer("Lara de Silva", 1000)
print(cust.name)
print(cust.balance)
```

```
The __init__ method was called
Lara de Silva
1000
```

Default arguments

```
class Customer:
    # Set a default value for balance
    def __init__(self, name, balance=0):
        self.name = name
        # Assign the new attribute
        self.balance = balance
        print("The __init__ method was called")
```

Default arguments

```
# Don't specify the balance explicitly
cust = Customer("Lara de Silva")
print(cust.name)
# The balance attribute is created anyway
print(cust.balance)
```

```
The __init__ method was called
Lara de Silva
0
```

Best practices

1. Initialize attributes in `__init__()`

Best practices

1. Initialize attributes in `__init__()`

2. Naming

`CamelCase` for classes, `lower_snake_case` for functions and attributes

Best practices

1. Initialize attributes in `__init__()`

2. Naming

`CamelCase` for class, `lower_snake_case` for functions and attributes

3. Keep `self` as `self`

```
class MyClass:
    # This works but isn't recommended
    def my_method(george, attr):
        george.attr = attr
```

Best practices

1. Initialize attributes in `__init__()`

2. Naming

`CamelCase` for class, `lower_snake_case` for functions and attributes

3. `self` is `self`

4. Use docstrings

```
class MyClass:
    """This class does nothing"""
    pass
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Class vs. instance attributes

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

Core principles of OOP

Encapsulation:

- Bundling of data and methods

Inheritance:

- Extending the functionality of existing code

Polymorphism:

- Creating a unified interface

Instance-level attributes

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

emp1 = Employee("Teo Mille", 50000)
emp2 = Employee("Marta Popov", 65000)
```

- `name` and `salary` values are specific to each object
- `self` assigns to an object

Class-level attributes

- Data shared among all instances of a class

- Define *class attributes* in the body of `class`

- "Global variable" within the class

Implementing class-level attributes

```
class Employee:
    # Define a class attribute
    # No self. syntax
    MIN_SALARY = 30000
    def __init__(self, name, salary):
        self.name = name
        # Use class name
        # to access class attribute
        if salary >= Employee.MIN_SALARY:
            self.salary = salary
        else:
            self.salary = Employee.MIN_SALARY
```

- `MIN_SALARY` is shared among all instances
- Don't use `self` to *define* class attribute
- Convention is to use capital letters
- Use `ClassName.ATTR_NAME` to *access* the class attribute value

Class-level attributes

```
class Employee:
    # Define a class attribute
    MIN_SALARY = 30000
    def __init__(self, name, salary):
        self.name = name
        # Use class name
        # to access class attribute
        if salary >= Employee.MIN_SALARY:
            self.salary = salary
        else:
            self.salary = Employee.MIN_SALARY
```

```
emp1 = Employee("John", 40000)
print(emp1.MIN_SALARY)
```

```
30000
```

```
emp2 = Employee("Jane", 60000)
print(emp2.MIN_SALARY)
```

```
30000
```

Modifying class-level attributes

```
emp1 = Employee("John", 40000)
emp2 = Employee("Jane", 60000)

# Update MIN_SALARY of emp1
emp1.MIN_SALARY = 50000

# Print MIN_SALARY for both employees
print(emp1.MIN_SALARY)
print(emp2.MIN_SALARY)
```

```
50000
30000
```

Modifying class-level attributes

- `MIN_SALARY` is created in the class definition

- Updating `MIN_SALARY` of an object will not affect the value in the class definition

- Security - prevent changes being made to software!

Why use class attributes?

Global constants related to the class

- Minimum and maximum values for attributes
  - Prevent invalid data
- Commonly used values and constants, e.g. `host` , `port` for a `Database` class
  - Avoid repetition when creating objects

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Class methods

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

Methods

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def give_raise(self, amount):
        self.salary += amount

# Unique attribute values
emp_one = Employee("John", 40000)
emp_one.give_raise(5000)
print(emp_one.salary)
```

```
45000
```

```
# Unique attribute values
emp_two = Employee("Jane", 60000)
emp_two.give_raise(5000)
print(emp_two.salary)
```

```
65000
```

Class methods

- Possible to define class methods
- Must have a narrow scope because they can't use object-level data

```
class MyClass:
    # Use a decorator to declare a class method
    @classmethod
    # cls argument refers to the class
    def my_awesome_method(cls, args...):
        # Do stuff here
        # Can't use any instance attributes
# Call the class, not the object
MyClass.my_awesome_method(args...)
```

- As with `self`, `cls` is a convention but any word will work

Alternative constructors

```
class Employee:
  def __init__(self, name, salary):
      self.name = name
      self.salary = salary
  @classmethod
  def from_file(cls, filename):
      with open(filename, "r") as f:
          # Read the first line
          name = f.readline().strip()
          # Read the second line as integer
          salary = int(f.readline().strip())
      return cls(name, salary)
```

- Allow alternative constructors
- Can only have one `__init__()`

- **Use class methods to create objects**
- Use `return` to return an object
- `cls(...)` will call `__init__(...)`

Alternative constructors

```
class Employee:
  def __init__(self, name, salary):
      self.name = name
      self.salary = salary
  @classmethod
  def from_file(cls, filename):
      with open(filename, "r") as f:
          name = f.readline().strip()
          salary = int(f.readline().strip())
      return cls(name, salary)
```

Employee.txt


```
# Create an employee without calling Employee()
emp = Employee.from_file("employee_data.txt")
print(emp.name)
```

```
John Smith
```

When to use class methods

- Alternative constructors
- Methods that don't require instance-level attributes
- Restricting to a single instance (object) of a class
  - Database connections
  - Configuration settings

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Class inheritance

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

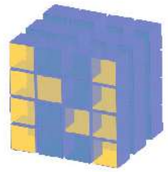
George Boorman
Curriculum Manager, DataCamp

datacamp

Code reuse

1. Someone has already done it

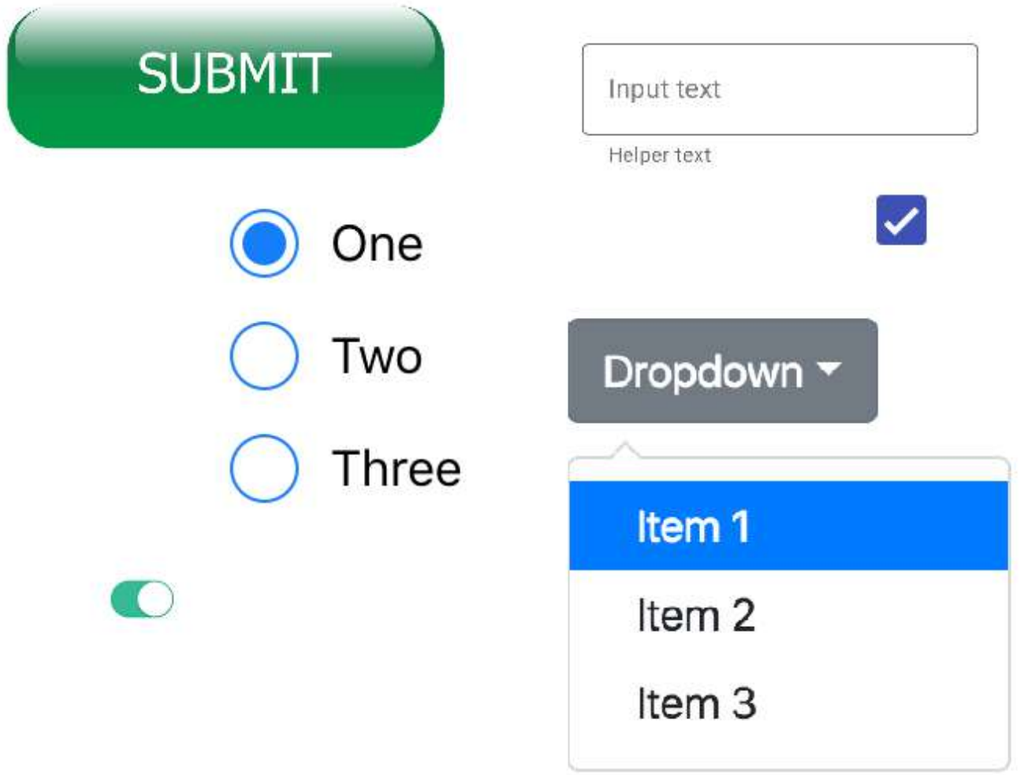
- Packages are great for fixed functionality
- OOP is great for customizing functionality



Code reuse

1. Someone has already done it

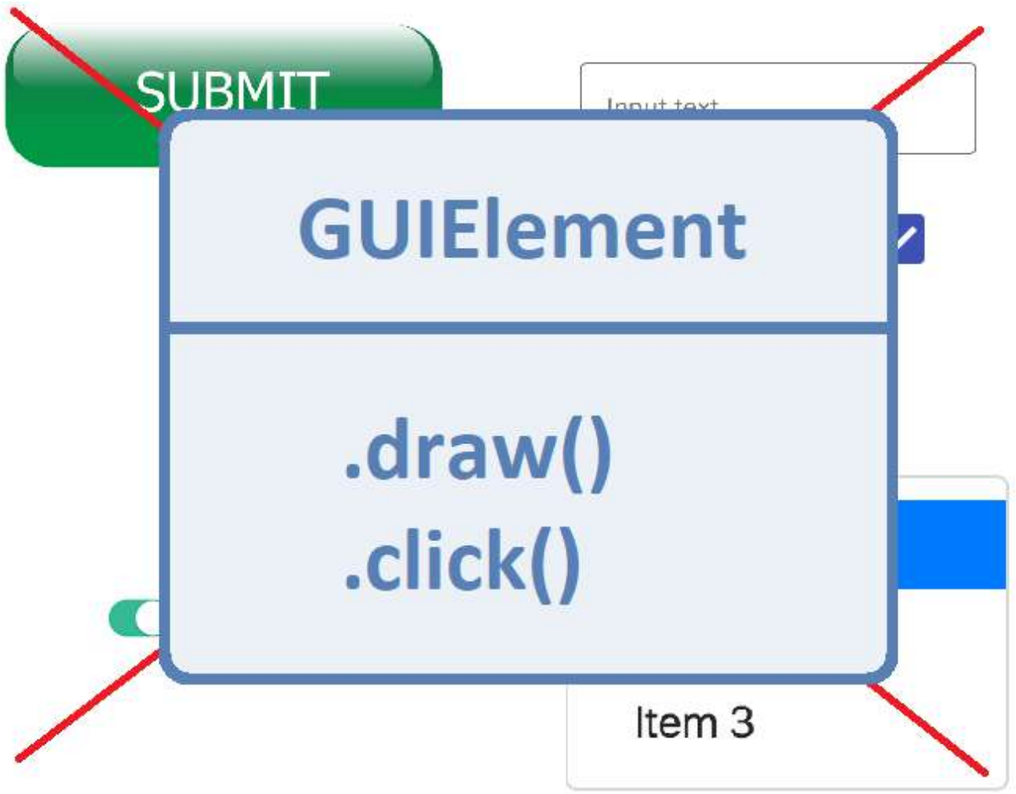
2. DRY: Don't Repeat Yourself



Code reuse

1. Someone has already done it

2. DRY: Don't Repeat Yourself

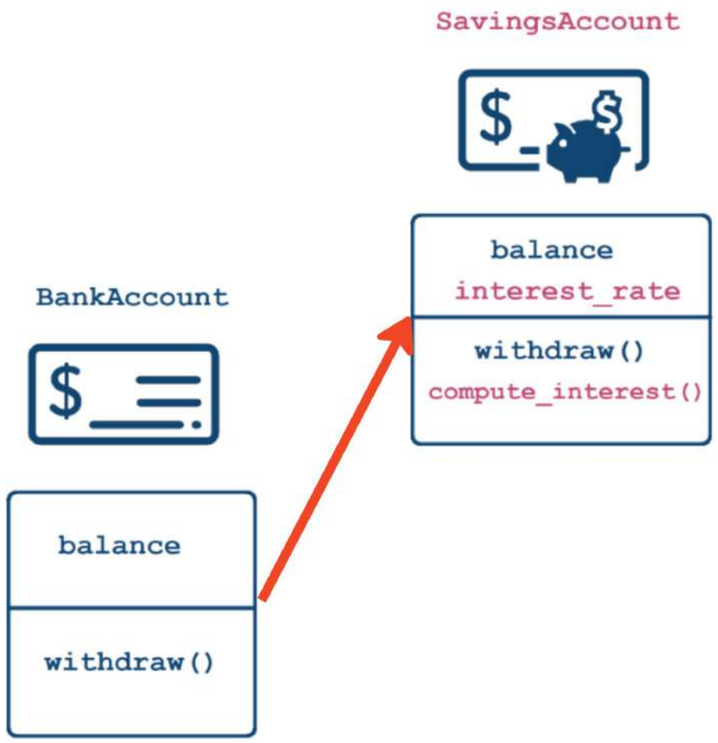


Inheritance

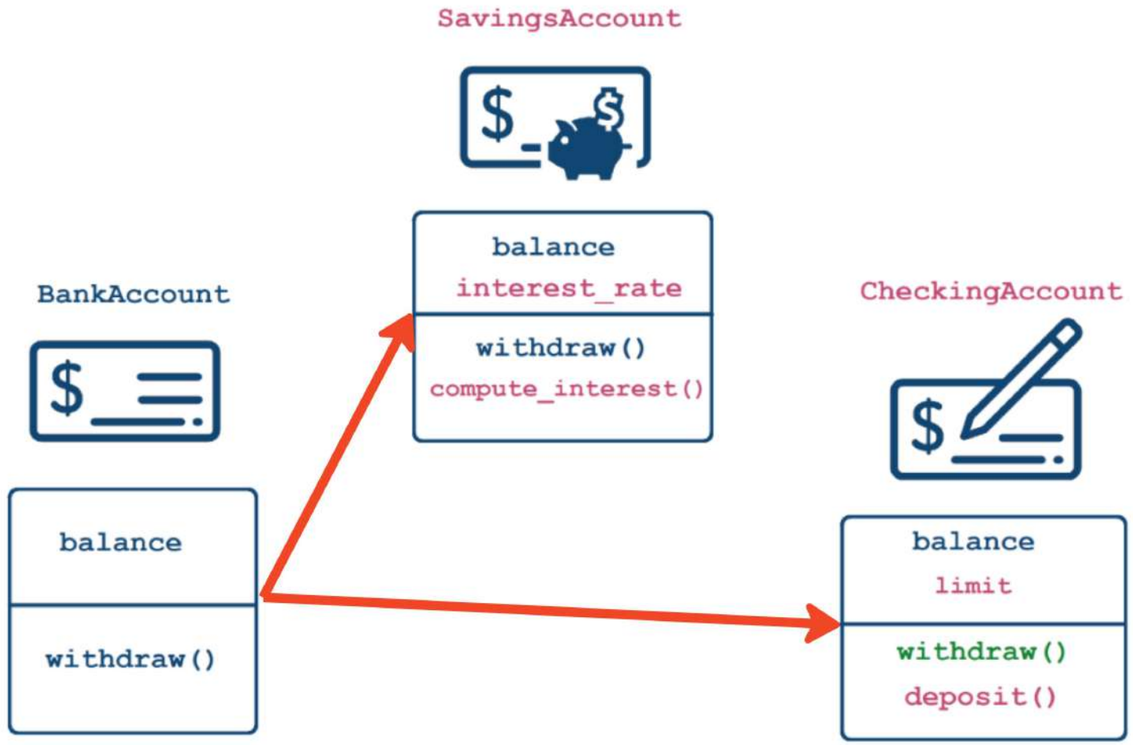
$$\text{New class functionality} = \text{Old class functionality} + \text{extra}$$

Example hierarchy

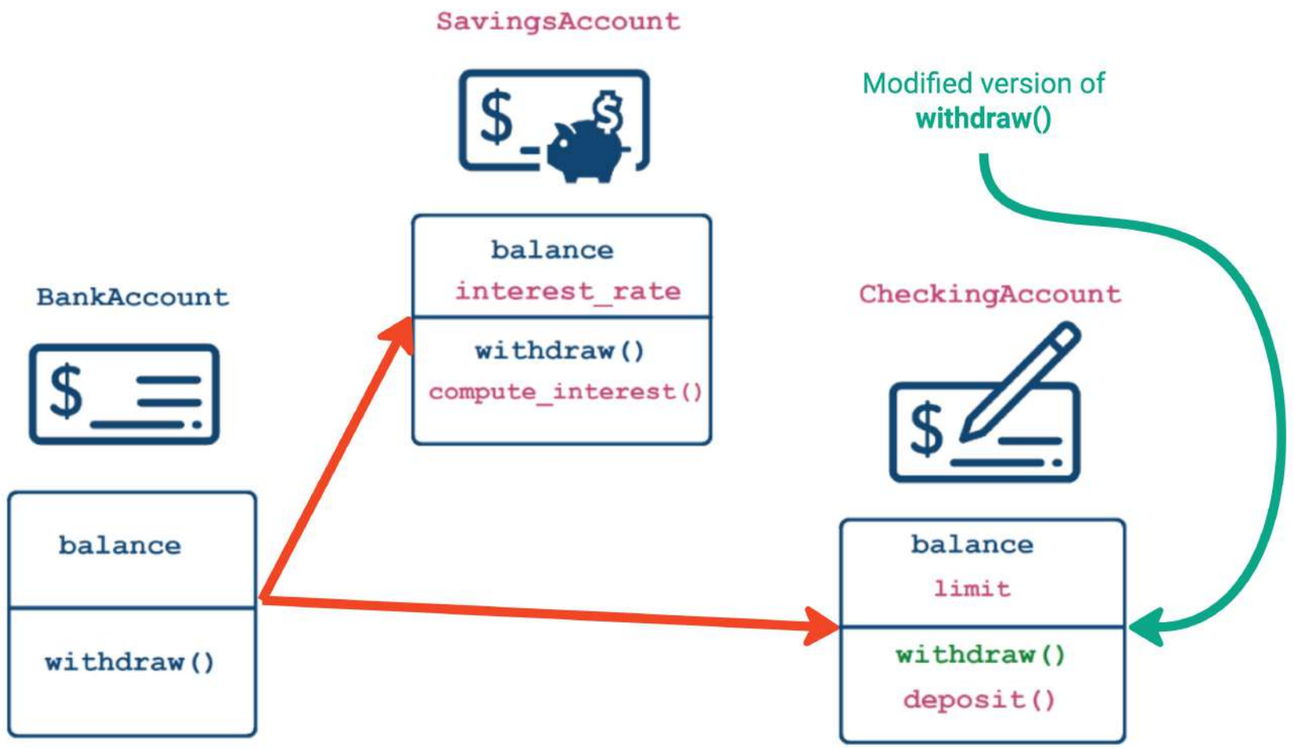
Example hierarchy



Example hierarchy



Example hierarchy



Implementing class inheritance

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        self.balance -= amount

# Class inheriting from BankAccount
class SavingsAccount(BankAccount):
    pass
```

- `BankAccount` : Parent class whose functionality is being extended/inherited
- `SavingsAccount` : Child/sub-class that will inherit the functionality and add more

Child class has all of the parent data

```
# Constructor inherited from BankAccount
savings_acct = SavingsAccount(1000)
type(savings_acct)
```

```
__main__.SavingsAccount
```

```
# Attribute inherited from BankAccount
savings_acct.balance
```

```
1000
```

```
# Method inherited from BankAccount
savings_acct.withdraw(300)
```

Inheritance: "is-a" relationship

A `SavingsAccount` is a `BankAccount`

(possibly with special features)

```
savings_acct = SavingsAccount(1000)
isinstance(savings_acct, SavingsAccount)
```

```
True
```

```
acct = BankAccount(500)
isinstance(acct, SavingsAccount)
```

```
False
```

```
isinstance(savings_acct, BankAccount)
```

```
True
```

```
isinstance(acct, BankAccount)
```

```
True
```

Let's practice!

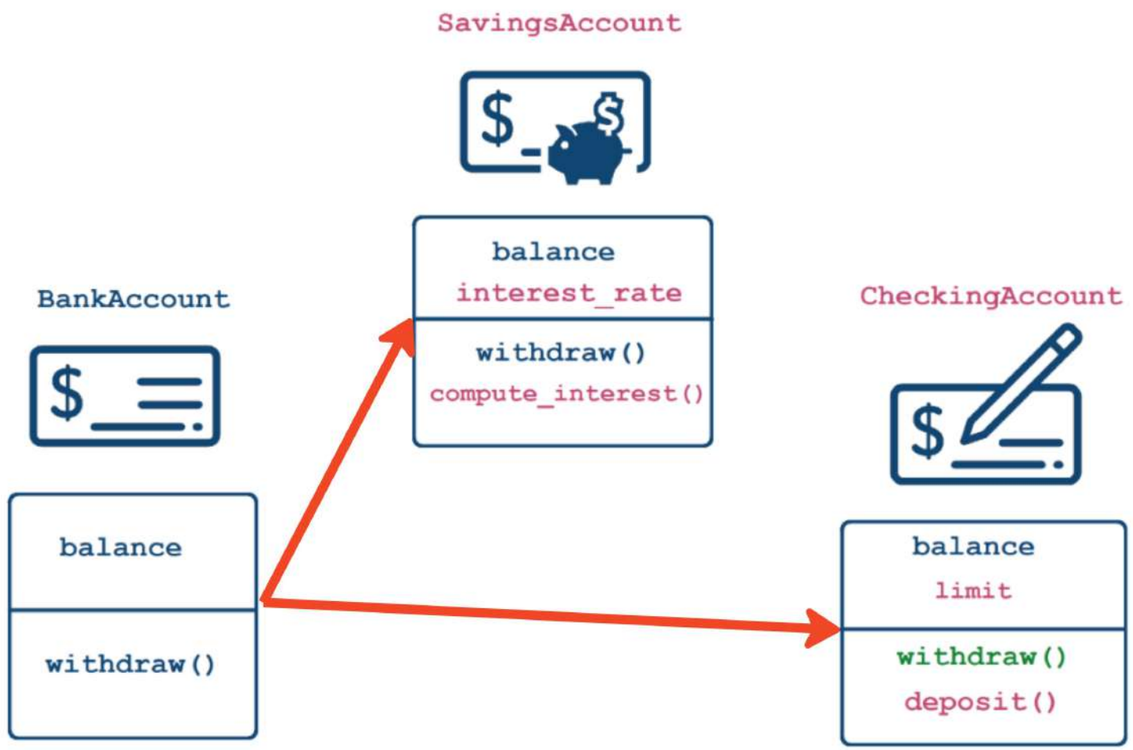
INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Customizing functionality via inheritance

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp


SavingsAccount
balance
interest_rate
withdraw()
compute_interest()
BankAccount
balance
withdraw()
CheckingAccount
balance
limit
withdraw()
deposit()

What we have so far

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        self.balance -=amount

# Empty class inherited from BankAccount
class SavingsAccount(BankAccount):
    pass
```

Customizing constructors

```
class SavingsAccount(BankAccount):
    # Constructor for SavingsAccount with an additional argument
    def __init__(self, balance, interest_rate):
        # Call the parent constructor using ClassName.__init__()
        # self is a SavingsAccount but also a BankAccount
        BankAccount.__init__(self, balance)
        # Add more functionality
        self.interest_rate = interest_rate
```

- Can run constructor of the parent class first by `Parent.__init__(self, args...)`
- Add more functionality
- Don't *have* to call the parent constructor

Create objects with a customized constructor

```
# Construct the object using the new constructor
acct = SavingsAccount(1000, 0.03)
acct.interest_rate
```

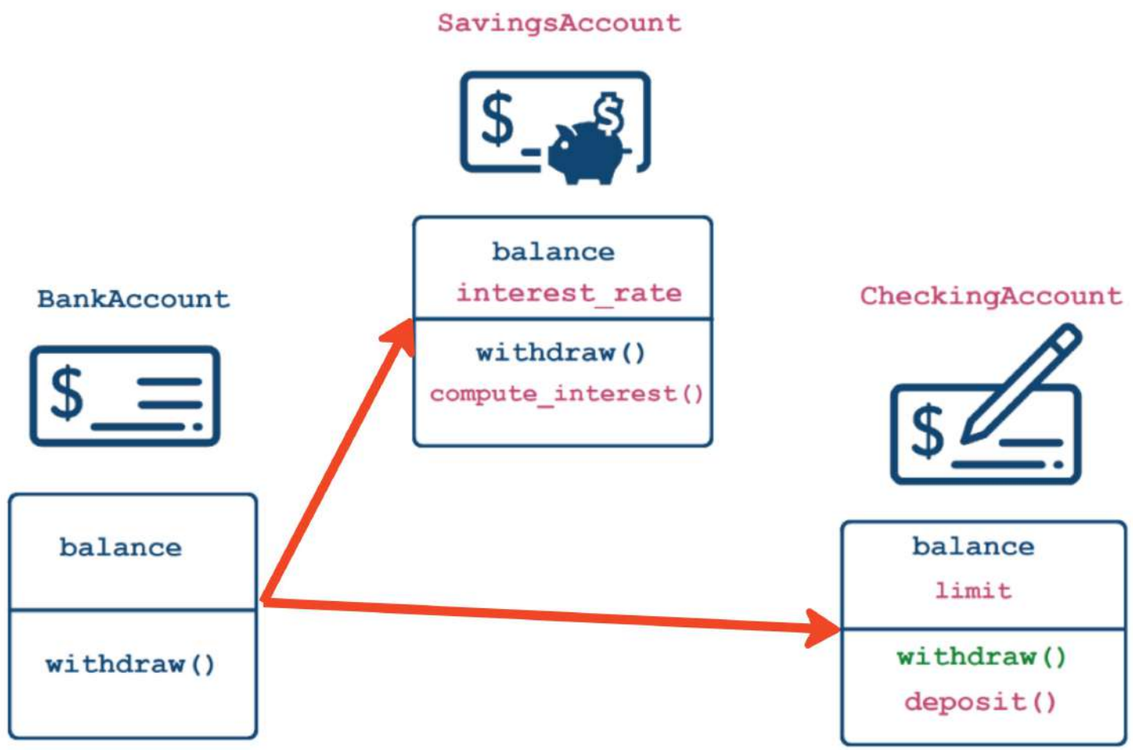
```
0.03
```

Adding functionality

- Add methods as usual
- Can use the data from both the parent and the child class

```
class SavingsAccount(BankAccount):
    def __init__(self, balance, interest_rate):
        BankAccount.__init__(self, balance)
        self.interest_rate = interest_rate

    # New functionality
    def compute_interest(self, n_periods=1):
        return self.balance * ( (1 + self.interest_rate) ** n_periods - 1)
```


SavingsAccount
balance
interest_rate
withdraw()
compute_interest()
BankAccount
balance
withdraw()
CheckingAccount
balance
limit
withdraw()
deposit()

Adding a second child class

```
class CheckingAccount(BankAccount):
    def __init__(self, balance, limit):
        BankAccount.__init__(self, balance) # Call the ParentClass constructor
        self.limit = limit
    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount, fee=0): # New fee argument
        if amount <= self.limit:
            BankAccount.withdraw(self, amount + fee)
        else:
            pass # Won't run if the condition isn't met
```

```
check_acct = CheckingAccount(1000, 25)

# Will call withdraw from CheckingAccount
check_acct.withdraw(200)

# Will call withdraw from CheckingAccount
check_acct.withdraw(200, fee=15)
```

```
bank_acct = BankAccount(1000)

# Will call withdraw from BankAccount
bank_acct.withdraw(200)

# Will produce an error
bank_acct.withdraw(200, fee=15)
```

```
TypeError: withdraw() got an unexpected keyword argument 'fee'
```

- Violates polymorphism
  - Parent / child classes have different methods

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Operator overloading: comparing objects

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

Object equality

```
class Customer:
    def __init__(self, name, balance):
        self.name, self.balance = name, balance

customer1 = Customer("Maryam Azar", 3000)
customer2 = Customer("Maryam Azar", 3000)

# Check for equality
customer1 == customer2
```

```
False
```

Object equality

```
class Customer:
    def __init__(self, name, balance, acc_id):
        self.name, self.balance = name, balance
        self.acc_id = acc_id

customer1 = Customer("Maryam Azar", 3000, 123)
customer2 = Customer("Maryam Azar", 3000, 123)
customer1 == customer2
```

```
False
```

Variables are references

```
customer_one = Customer("Maryam Azar", 3000, 123)
customer_two = Customer("Maryam Azar", 3000, 123)
print(customer_one)
```

```
<__main__.Customer at 0x1f8598e2e48>
```

```
print(customer_two)
```

```
<__main__.Customer at 0x1f8598e2240>
```

- `print()` output refers to the memory chunk that the variable is assigned to
- `==` compares references, not data

Custom comparison

```
# Two different lists containing the same data
list_one = [1,2,3]
list_two = [1,2,3]


list_one == list_two
```

```
True
```

The __eq__() method

- `__eq__()` is called when 2 objects of a class are compared using `==`
- Accepts 2 arguments, `self` and `other` - objects to compare
- Returns a Boolean

The __eq__() method

```
class Customer:
    def __init__(self, acc_id, name):
        self.acc_id, self.name = acc_id, name
    # Will be called when == is used
    def __eq__(self, other):
        # Printout
        print("__eq__() is called")

        # Returns True if all attributes match
        return (self.acc_id == other.acc_id) and (self.name == other.name)
```

Comparison of objects

```
# Two equal objects
customer1 = Customer(123, "Maryam Azar")
customer2 = Customer(123, "Maryam Azar")

customer1 == customer2
```

```
__eq__() is called
True
```

```
# Two unequal objects - different ids
customer1 = Customer(123, "Maryam Azar")
customer2 = Customer(456, "Maryam Azar")

customer1 == customer2
```

```
__eq__() is called
False
```

Checking types

- What if two objects of different classes have the same attributes and values?
  - Python will evaluate them as equal

```
class Customer:
    def __init__(self, acc_id, name):
        self.acc_id, self.name = idacc_id name

    def __eq__(self, other):
        # Returns True if the objects have the same attributes
        # and are of the same type
        return (self.acc_id == other.acc_id) and (self.name == other.name)\
            and (type(self) == type(other))
```

Other comparison operators

Operator	Method
`==`	`__eq__()`
`!=`	`__ne__()`
`>=`	`__ge__()`
`<=`	`__le__()`
`>`	`__gt__()`
`<`	`__lt__()`

- Customize by defining within a class

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Inheritance comparison and string representation

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

Comparing objects from different classes

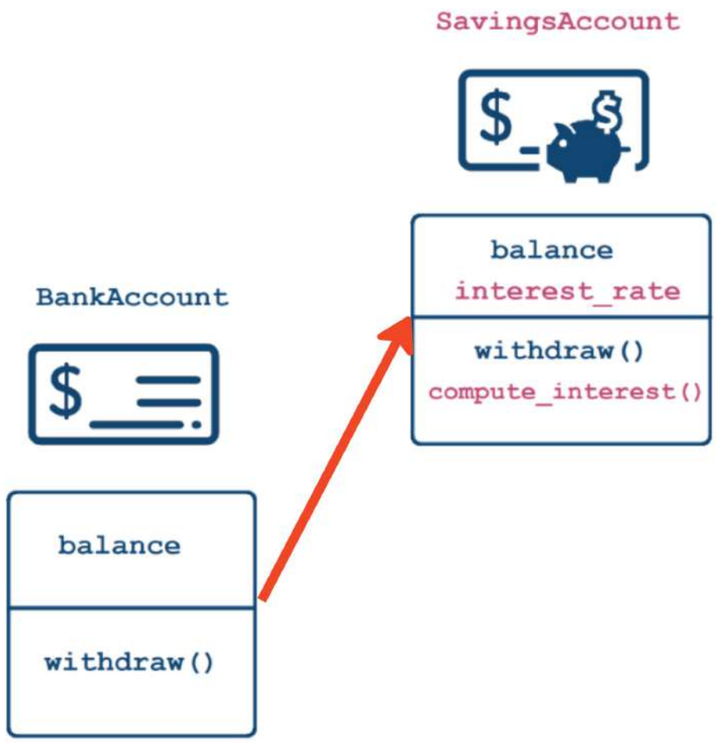
```
class Customer:
    def __init__(self, acc_id, name):
        self.acc_id, self.name = acc_id, name

    def __eq__(self, other):
        # Returns True if the objects have the same attributes
        # and are of the same type
        return (self.acc_id == other.acc_id) and (self.name == other.name)\
            and (type(self) == type(other))
```

Comparing objects with inheritance

What if one object *inherits* from the class of the other object?

Bank and savings accounts



__eq__ in parent/child classes

```
class BankAccount:
    def __init__(self, number, balance=0):
        self.balance = balance
        self.number = number

    def withdraw(self, amount):
        self.balance -= amount

    # Define __eq__ that returns True
    # if the number attributes are equal
    def __eq__(self, other):
        print("BankAccount __eq__() called")
        return self.number == other.number
```

```
class SavingsAccount(BankAccount):
    def __init__(self, number, balance, interes
        BankAccount.__init__(self, number,
                             balance)
          self.interest_rate = interest_rate

    # Define __eq__ that returns True
    # if the number attributes are equal
    def __eq__(self, other):
        print("SavingsAccount __eq__() called")
        return self.number == other.number
```

Comparing parent and child objects

```
ba = BankAccount(123, 10000)
sa = SavingsAccount(456, 2000, 0.05)
# Compare the two objects
ba == sa
```

```
SavingsAccount __eq__() called
False
```

```
sa == ba
```

```
SavingsAccount __eq__() called
False
```

Printing an object

```
class Customer:
    def __init__(self, name, balance):
        self.name, self.balance = name, balance

cust = Customer("Maryam Azar", 3000)
print(cust)
```

```
<__main__.Customer at 0x1f8598e2240>
```

```
a_list = [1,2,3]
print(a_list)
```

```
[1, 2, 3]
```

`__str__()`

- `print(obj)`, `str(obj)`

```
print(np.array([1,2,3]))
```

```
[1 2 3]
```

```
str(np.array([1,2,3]))
```

```
'[1 2 3]'
```

- informal, for end user
- ***str***ing representation

`__repr__()`

- `repr(obj)`, printing in console

```
repr(np.array([1,2,3]))
```

```
'array([1,2,3])'
```

```
np.array([1,2,3])
```

```
array([1, 2, 3])
```

- formal, for developer
- ***repr***oducible ***repr***esentation
- fallback for `print()`

Implementation: repr

```
class Customer:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def __repr__(self):
        # Notice the '...' around name
        return f"Customer('{self.name}', {self.balance})"

cust = Customer("Maryam Azar", 3000)
# Will implicitly call __repr__()
cust
```

```
Customer('Maryam Azar', 3000)
```

Implementation: str

```
class Customer:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
    def __str__(self):
        cust_str = f"""
        Customer:
            name: {self.name}
            balance: {self.balance}
            """
        return cust_str
```

```
cust = Customer("Maryam Azar", 3000)

# Will implicitly call __str__()
print(cust)
```

```
Customer:
  name: Maryam Azar
  balance: 3000
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Exceptions

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

```
a = 1
a / 0
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    1/0
ZeroDivisionError: division by zero
```

```
a = [1,2,3]
a[5]
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    a[5]
IndexError: list index out of range
```

```
a = 1
a + "Hello"
```

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    a + "Hello"
TypeError: unsupported operand type(s) for +: /
'int' and 'str'
```

```
a = 1
a + b
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    a + b
NameError: name 'b' is not defined
```

Exception handling

- Prevent the program from terminating when an exception is raised
- `try` - `except` - `finally`:

```
try:
    print(5 + "a")
except TypeError:
    print("You can't add an integer to a string, but you can multiply them!")
# Can have multiple except blocks
except AnotherExceptionHere:
    # Run this code if AnotherExceptionHere happens
# Optional finally block
finally:
    print(5 * "a")
```

Exception handling output

```
You can't add an integer to a string, but you can multiply them!
aaaaa
```

Raising exceptions

```
def make_list_of_ones(length):
    if length <= 0:
        # Custom message if ValueError occurs
        # Will stop the program and raise the error
       raise ValueError("Invalid length!")
    return [1]*length

make_list_of_ones(-1)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    make_list_of_ones(-1)
  File "<stdin>", line 3, in make_list_of_ones
    raise ValueError("Invalid length!")
ValueError: Invalid length!
```

Exceptions are classes

- Exceptions are inherited from `BaseException` or `Exception`

```
BaseException
+-- Exception
    +-- ArithmeticError
    |    +-- FloatingPointError
    |    +-- OverflowError
    |    +-- ZeroDivisionError
    +-- TypeError
    +-- ValueError
    |    +-- UnicodeError
    |         +-- UnicodeDecodeError
    |         +-- UnicodeEncodeError
    |         +-- UnicodeTranslateError
    +-- RuntimeError
   ...
+-- SystemExit
...
```

[1] https://docs.python.org/3/library/exceptions.html

Custom exceptions

- Inherit from `Exception` or one of its subclasses
- Usually an empty class

```
class BalanceError(Exception):
    pass

class Customer:
    def __init__(self, name, balance):
        if balance < 0 :
            raise BalanceError("Balance has to be non-negative!")
        else:
            self.name = name
            self.balance = balance
```

Exception in constructor

```
cust = Customer("Larry Torres", -100)
```

```
Traceback (most recent call last):
  File "script.py", line 11, in <module>
    cust = Customer("Larry Torres", -100)
  File "script.py", line 6, in __init__
    raise BalanceError("Balance has to be non-negative!")
BalanceError: Balance has to be non-negative!
```

Exceptions terminate the program

- Exception interrupted the constructor → object not created

```
cust
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    cust
NameError: name 'cust' is not defined
```

Catching custom exceptions

```
try:
    cust = Customer("Larry Torres", -100)
except BalanceError:
    cust = Customer("Larry Torres", 0)
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

Congratulations

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

George Boorman
Curriculum Manager, DataCamp

datacamp

Classes and objects

Term	Definition
Class	A **blueprint**/template used to build objects
Object	A combination of *data* and *functionality*; An **instance** of a class

Attributes and methods

Term	Definition
Class	A **blueprint**/template used to build objects
Object	A combination of *data* and *functionality*; An **instance** of a class
State	*Data* associated with an object, assigned through **attributes**
Behavior	An object's *functionality*, defined through **methods**

Core concepts

Encapsulation:

- Bundling of data and methods

Inheritance:

- Extending the functionality of existing code

Polymorphism:

- Creating a unified interface

Comparisons

Operator	Method
`==`	`__eq__()`
`!=`	`__ne__()`
`>=`	`__ge__()`
`<=`	`__le__()`
`>`	`__gt__()`
`<`	`__lt__()`

String representation

`__str__()`

- `print(obj)`, `str(obj)`

```
print([1,2,3])
```

```
[1 2 3]
```

```
str([1,2,3])
```

```
'[1, 2, 3]'
```

- informal, for end user
- ***str***ing representation

`__repr__()`

- `repr(obj)`, printing in console

```
repr([1,2,3])
```

```
[1,2,3]
```

```
[1,2,3]
```

```
[1,2,3]
```

- formal, for developer
- ***repr***oducible ***repr***esentation

Error-handling

```
class BalanceError(Exception):
    pass

class Customer:
    def __init__(self, name, balance):
        if balance < 0 :
            raise BalanceError("Balance has to be non-negative!")
        else:
            self.name, self.balance = name, balance
# Use try-except to catch errors
try:
    cust = Customer("Larry Torres", -100)
except BalanceError:
    cust = Customer("Larry Torres", 0)
```

Where to next?

- Multiple inheritance
- Descriptors
- Custom attributes
- Custom iterators
- Type hints
- Abstract base classes

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON